Guion de la presentación · InfoSport Agente

Texto para leer mientras pasas las diapositivas de *presentacion_infosport_agente.pdf* · duración total ≈ 10 min

A · PRODUCTO · ≈ 4 min

Diapositiva 1 · Portada (15 s)

Hola, soy Yordanos López. Os presento **InfoSport Agente**, mi Project Break 2.

Es un asistente multiagente sobre el deporte municipal de Madrid, construido con LangGraph, que reutiliza el RAG que hice en el Project Break 1.

Diapositiva 2 · El problema (40 s)

Si eres usuario de los polideportivos municipales de Madrid, la información que necesitas está repartida en sitios distintos y en formatos distintos.

Las normas están en un reglamento en PDF de veinticinco páginas. Las clasificaciones y los partidos de los Juegos Deportivos Municipales están en ficheros CSV con miles de filas. Los horarios y la dirección de cada centro están en el portal de datos abiertos. Y además influyen cosas como el tiempo que hace o lo que pone en un cartel.

Un chatbot genérico se inventa la respuesta, y un buscador te obliga a abrir cuatro webs distintas.

Diapositiva 3 · La solución: un equipo de agentes (45 s)

InfoSport Agente funciona como un equipo. Un **supervisor** entiende lo que le pides y reparte el trabajo entre **tres especialistas**: uno de normativa, uno de instalaciones y uno de imágenes.

Cada especialista consulta **fuentes reales** mediante herramientas. Si un dato no está en las fuentes, el agente lo dice en lugar de inventárselo.

Al final te propone un plan, y eres **tú quien decide** si lo aprueba o lo rechaza. Es lo que se llama human in the loop.

Por ejemplo, si pregunto a qué hora cierra la taquilla del CDM La Elipa y si hace buen tiempo para correr, consulta la API de polideportivos y la del tiempo, y me responde con los dos datos y un plan.

Diapositiva 4 · Dominio y datos (35 s)

Todos los datos son públicos y de Madrid: el reglamento oficial de instalaciones deportivas, las clasificaciones y los partidos de la temporada 2025-2026, la API de centros deportivos del Ayuntamiento y Open-Meteo para el tiempo.

Elegí este dominio porque continúa el Project Break 1, con el mismo corpus, y porque mezcla texto normativo, tablas y datos en vivo, que es justo lo que obliga a orquestar varias herramientas.

Diapositiva 5 · Pieza 2 de un puzzle (25 s)

Este proyecto es la segunda pieza de tres. La primera fue el RAG. Esta segunda es el agente, y la tercera será empaquetarlo, desplegarlo y monitorizarlo con MLOps.

La idea clave es que el agente **no sustituye al RAG**: lo llama como una herramienta más, a través de su función *responder*, que funciona sin ninguna interfaz.

Diapositiva 6 · Demo de uso (50 s)

Esta es la aplicación web en Streamlit.

Pregunto a qué hora cierra la taquilla del CDM Moratalaz, cómo llego en metro y qué obligaciones tengo como usuario. Es un pedido mixto: el supervisor lo reparte entre el especialista de instalaciones, que consulta la API de polideportivos, y el de normativa, que consulta el RAG del reglamento.

La respuesta indica de dónde sale cada dato. En la barra lateral aparece el plan con los botones **Aprobar** y **Rechazar**, y el chat queda bloqueado hasta que decido. Si apruebo, la conversación se cierra; si rechazo, el plan se descarta y puedo pedir otra cosa.

B · TÉCNICO · ≈ 6 min

Diapositiva 7 · Arquitectura: grafo multiagente (50 s)

Por dentro es un **StateGraph** de LangGraph.

Todo empieza en el supervisor, que elige a qué especialista llamar. Cada especialista hace su trabajo y vuelve al supervisor con sus notas. Cuando ya hay información suficiente, pasa al nodo **sintetizar**, que escribe la respuesta y el plan.

Con el plan listo, el grafo va al nodo **aplicar**. Ahí está el **interrupt before**: el grafo se detiene hasta que decide una persona.

Lo importante es que las reglas están en Python y no en el modelo: el supervisor no puede elegir un nodo que no existe ni repetir especialista, hay un máximo de seis vueltas y solo el nodo aplicar puede marcar la conversación como terminada.

Diapositiva 8 · Tools controladas: allowlist y traza (40 s)

El agente tiene seis herramientas: el RAG, dos APIs externas (polideportivos y tiempo), una consulta exacta de partidos sobre el CSV, una herramienta auxiliar de hora y la herramienta multimodal de imágenes.

El modelo solo **propone** qué herramienta usar y con qué argumentos. Es Python quien decide: la función *ejecutar tool* comprueba que esté en la allowlist general y en la del especialista. Si no lo está, queda bloqueada. Si falla, el error queda registrado y el grafo sigue funcionando.

Cada llamada deja una traza con el nombre, los argumentos, el estado y los milisegundos.

Diapositiva 9 · Un pedido, dos especialistas, traza completa (35 s)

Este es el panel de debug para el pedido de la demo.

Se ve el recorrido completo: el supervisor manda primero al especialista de instalaciones, que llama a buscar polideportivos; después al de normativa, que llama al RAG; y luego a sintetizar, que deja el plan en pausa.

Arriba se ven los flags: el thread id de la conversación, que hay un HITL pendiente y que todavía no está terminada.

Diapositiva 10 · El RAG del PB1 como tool (45 s)

El RAG es el del Project Break 1, con mejoras. Con el comando *prepare index* se cargan los documentos, se trocean, se generan los embeddings y se guardan en Chroma: 1.674 fragmentos en unos ochenta segundos.

Al preguntar por la multa máxima de una infracción muy grave responde «hasta 3.000 euros», citando el artículo 60 del reglamento. Y si pregunto algo que no está en el corpus, como las calorías que se queman en las barcas, **se abstiene**: dice que no tiene evidencia.

Respecto al PB1, agrupé las clasificaciones por grupo, saqué los partidos a una herramienta exacta y usé embeddings locales que no dependen de cuota.

Diapositiva 11 · HITL y persistencia (45 s)

El human in the loop funciona así. La función *procesar turno* ejecuta el grafo con un thread id, y el grafo se para justo antes de aplicar. Cuando la persona decide, *aprobar plan* guarda la decisión con *update state* y reanuda el grafo.

Si apruebo, la conversación queda terminada. Si rechazo, se descartan el plan y las notas, pero el hilo sigue abierto para otro pedido.

La memoria es un **MemorySaver**: con el mismo thread id se recupera la conversación. Como extra, también funciona con **SqliteSaver**, y entonces un plan pendiente sobrevive aunque cierre el programa.

Diapositiva 12 · Multimodal: imagen a texto (35 s)

La herramienta multimodal usa Gemini con visión.

La imagen de demo es una ficha del Centro Deportivo Municipal Moratalaz, generada con los datos reales del portal abierto. Al preguntar a qué hora cierra la taquilla el sábado, la herramienta transcribe la ficha y responde «a las 14:30», sin inventar nada que no aparezca en la imagen.

Por seguridad, solo puede leer imágenes de la carpeta del proyecto.

Diapositiva 13 · Límites y robustez (35 s)

Todos los límites están en el fichero de configuración y se aplican en Python: máximo de vueltas del supervisor, de llamadas por especialista y de turnos por conversación.

Además hay varias redes de seguridad. Si el modelo falla, el supervisor decide por palabras clave y cada especialista llama a su herramienta principal. Si un modelo se queda sin cuota, se pasa al siguiente de una cadena de cinco. Y si no hay red, las APIs usan una copia local, que queda marcada en la traza.

Diapositiva 14 · Git y Spec-Driven Development (30 s)

En Git trabajé con una rama main estable, una rama develop de integración y una rama por cada bloque de trabajo, integradas con merge.

Antes de programar cada parte escribí su especificación en la carpeta *specs* y la usé como guía para implementarla. Al final comprobé todo desde un clon limpio del repositorio. Y no hay secretos en el repo: la clave está en el fichero punto env, que git ignora.

Diapositiva 15 · Tres fallos reales y siguientes pasos (50 s)

Os cuento tres fallos reales.

Primero, la cuota diaria de embeddings de Gemini se agotó a mitad del indexado. Lo resolví con embeddings locales, y ahora indexa en ochenta segundos. En MLOps, el índice sería un artefacto versionado.

Segundo, los modelos saturados o sin cuota en mitad de la demo. Lo resolví con la cadena de modelos y los fallbacks. En MLOps añadiría métricas y alertas por modelo.

Tercero, el entorno Windows: certificados SSL y una librería de Visual C++ antigua que hacía fallar la base de datos vectorial. La solución definitiva es un contenedor Docker, que es precisamente la tercera pieza.

Diapositiva 16 · Cierre (15 s)

Todo el proyecto está en el repositorio de GitHub, con el README, las capturas, el informe de decisiones y las especificaciones.

Muchas gracias. ¿Alguna pregunta?